Redes Neuronales Recurrentes, LSTM y Series Temporales

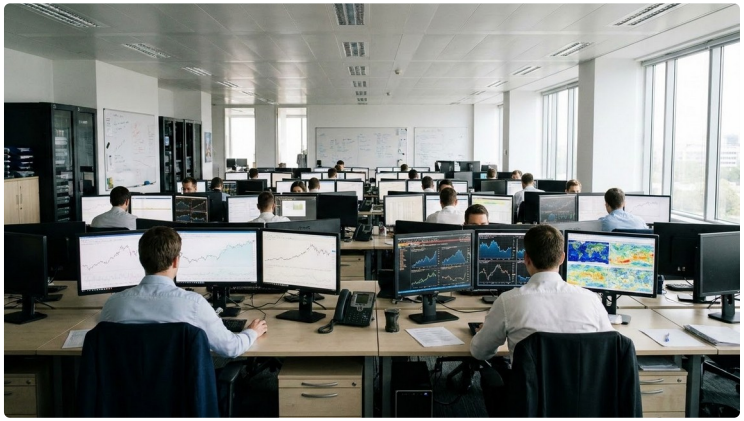
Semana 11 — Inteligencia Artificial

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática

Docentes: Prof. Hugo Vega / Dr. David Mauricio

Estudiante: Gonzalo Manuel Calero Falcon | Semestre 2026-II



Contenido de la exposición

1. ¿Por qué las redes anteriores fallan con secuencias y el tiempo?: limitaciones del Perceptrón Multicapa y las redes convolucionales ante dependencias temporales.
2. Introducción a las Redes Recurrentes (RNN) y el Estado Oculto: formulación matemática, vectores de memoria y el principio de recurrencia temporal.
3. Entrenamiento en el tiempo (BPTT) y el problema de la amnesia: algoritmo Backpropagation Through Time y desvanecimiento de gradientes.
4. Celdas con memoria a largo plazo: Arquitecturas LSTM y GRU: compuertas lógicas, canales aditivos y reducción de parámetros computacionales.
5. Preparación de ventanas deslizantes y código en Python con Keras: estructuración de tensores 3D, prevención del sesgo temporal e implementación funcional.
6. Caso de estudio: Predicción bursátil y volatilidad en la UNMSM: aplicaciones de vanguardia basadas en las publicaciones de Mauricio et al. (2025).

¿Qué les faltaba a las redes anteriores?

En la Semana 9 estudiamos redes para tablas de datos (MLP) y en la Semana 10 redes para fotografías (CNN). Aunque ambas son muy potentes, tienen una limitación común: no tienen memoria del tiempo. Reciben un dato o una imagen en un instante, entregan su respuesta e inmediatamente olvidan todo lo que acaban de ver, asumiendo que cada ejemplo es independiente del anterior.

En muchos problemas de la vida real, el orden del tiempo cambia por completo el significado de las cosas. Por ejemplo, en las frases 'el banco cerró porque el río creció' frente a 'el banco cerró porque era feriado', necesitamos recordar las demás palabras para saber de qué banco se habla. De igual forma, para predecir el clima de mañana o el precio de una acción, es indispensable recordar cómo vino cambiando en los días previos (Goodfellow et al., 2016).



Monitoreo en tiempo real: En secuencias fisiológicas y señales analógicas, cada pulso depende del historial inmediato previo para poseer validez diagnóstica.

¿Qué es una Red Neuronal Recurrente?

Para darle memoria del pasado a una neurona artificial, en 1990 Jeffrey Elman propuso la Red Neuronal Recurrente (RNN) agregándole un bucle de conexión sobre sí misma. Podemos imaginar este mecanismo como una neurona que tiene una libreta de apuntes interna llamada Estado Oculto (h_t), donde va guardando un resumen de lo que ha visto hasta ese momento.

Cuando llega el dato del lunes, la neurona lo procesa y escribe un resumen en su estado oculto. Cuando llega el martes, la neurona no empieza desde cero: lee el dato nuevo del martes y al mismo tiempo revisa el apunte que dejó el lunes para actualizar su memoria hacia el miércoles. Además, la misma neurona utiliza exactamente los mismos pesos en todos los días de la secuencia (Chollet, 2021).

Despliegue temporal de la recurrencia (BPTT)



— Avance secuencial: Transmisión del estado oculto `h_t` (libreta de apuntes) conservando los mismos pesos compartidos `W` en cada jornada.

- - Retroalimentación BPTT: Propagación hacia atrás del error a través del horizonte temporal para ajustar la memoria.

Cálculo interno de la memoria en una RNN

En cada paso de tiempo t, una celda RNN realiza dos operaciones sencillas. Primero actualiza su memoria interna (el estado oculto h_t) sumando dos partes: lo que recordaba del paso anterior multiplicado por sus pesos de memoria, más el dato nuevo de hoy multiplicado por sus pesos de entrada: `h_t = tanh(W_hh · h_(t-1) + W_xh · x_t + b)`. Luego, con esa memoria actualizada, calcula la predicción del día.

Un detalle muy importante es por qué aquí se utiliza la función tangente hiperbólica (tanh). Como en una serie de tiempo la neurona repite la misma multiplicación durante 30 o 50 días seguidos, si usáramos números sin límite el resultado crecería hasta el infinito; en cambio, la función tanh mantiene todos los valores de la memoria siempre ordenados entre -1 y +1.



Estación de trabajo analítica: evaluación empírica y ajuste matemático del estado recurrente en series temporales.

Conexión de entradas y salidas en el tiempo

Gracias a su capacidad de procesar secuencias paso a paso, las redes recurrentes se pueden configurar de cuatro maneras según el problema (Russell & Norvig, 2021). La primera es 'Uno a Muchos', donde entra una sola fotografía y la red genera una frase palabra por palabra describiendo la escena. La segunda es 'Muchos a Uno', donde entran varios datos pasados (como los últimos 30 días de temperaturas) y la red entrega un solo valor final (el pronóstico de mañana).

Las otras dos configuraciones son de tipo 'Muchos a Muchos'. Puede ser sincronizada en el tiempo (por ejemplo, cuando entra una señal médica y en cada segundo sale una alerta), o puede ser para traducción de idiomas (Seq2Seq), donde la red primero lee toda una oración en español y luego escribe la oración equivalente en quechua o en inglés.

1. Uno a Muchos

Entrada: 1 imagen médica o satelital.

Salida: Secuencia de palabras descriptivas generadas secuencialmente.

2. Muchos a Uno

Entrada: Ventana temporal histórica de 30 días.

Salida: Pronóstico escalar puntual del día siguiente.

3. Muchos a Muchos (Sinc)

Entrada: Señal de electrocardiograma por segundo.

Salida: Etiqueta de anomalía en tiempo real para cada instante.

4. Muchos a Muchos (Seq2Seq)

Codificador-Decodificador:

Traducción oracional completa de español a quechua o inglés.

Aplicaciones en el Perú y algoritmo BPTT

En nuestro país, los modelos secuenciales tienen aplicaciones directas en sectores clave: en el SENAMHI se emplean para pronosticar heladas agrícolas en la sierra y crecidas de ríos analizando las semanas previas; en el COES sirven para estimar cuánta energía eléctrica consumirá el país mañana hora por hora; y en la Bolsa de Valores de Lima permiten proyectar precios de acciones y el tipo de cambio.

Para aprender a hacer estos pronósticos, la RNN utiliza un algoritmo llamado Retropropagación a través del Tiempo (BPTT). Funciona con la misma lógica que vimos en la Semana 9, pero como aquí los datos están encadenados por fechas, el error calculado al final tiene que viajar hacia atrás día por día desde el último paso hasta el primer día de la serie para corregir los pesos.

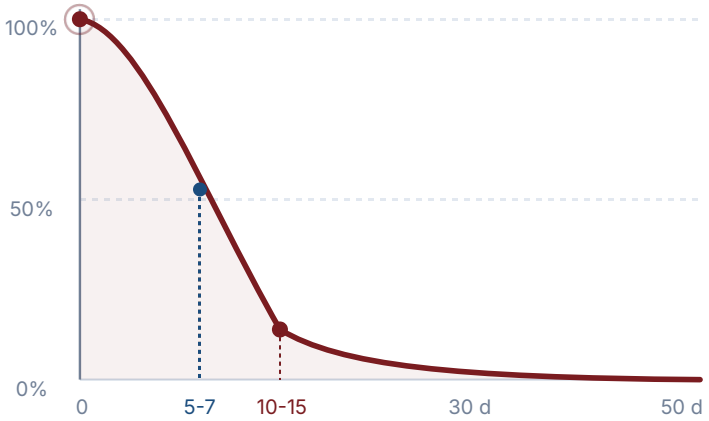


Monitoreo agrometeorológico continuo en zonas andinas para mitigación de heladas y gestión de recursos hídricos.

El problema de la amnesia a largo plazo

Al entrenar las primeras redes recurrentes con secuencias de varias semanas, Sepp Hochreiter (1991) y Yoshua Bengio (1994) descubrieron una falla grave. Cuando el error viaja hacia atrás día tras día mediante el algoritmo BPTT, en cada paso se va multiplicando por la derivada de la función tanh y por los pesos recurrentes, que suelen ser números decimales menores a uno (por ejemplo, 0.8).

Si multiplicamos 0.8 diez veces obtenemos 0.10, pero si retrocedemos 50 días (0.8 elevado a la 50), el resultado cae a 0.000014, es decir, prácticamente cero. Este fenómeno se conoce como desvanecimiento del gradiente y provoca que la RNN simple sufra de amnesia a largo plazo: recuerda bien lo que ocurrió en los últimos 5 o 7 días, pero olvida por completo los eventos de hace un mes.



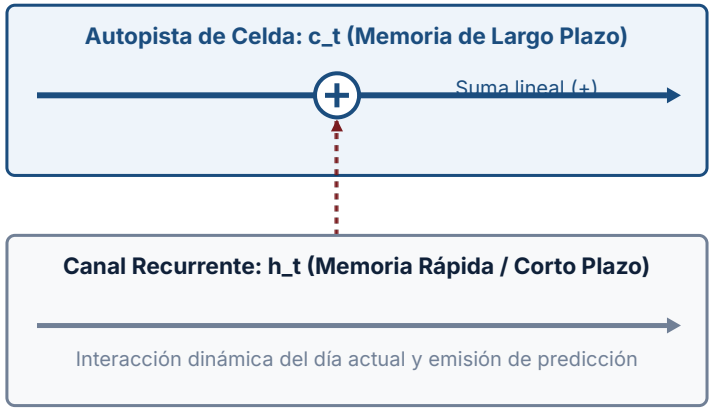
Desvanecimiento del gradiente en BPTT:

La retención cae del 100% a valores prácticamente nulos tras superar 10 o 15 pasos. A los 50 días retrocedidos ($0.8^{50} \approx 0.000014$), el gradiente se extingue e impide el aprendizaje a largo plazo.

La arquitectura LSTM

Para solucionar la amnesia de las redes recurrentes simples, en 1997 Sepp Hochreiter y Jürgen Schmidhuber diseñaron la celda LSTM (Long Short-Term Memory, o Memoria a Corto y Largo Plazo). Ellos notaron que el recuerdo se borraba en la RNN básica porque había un solo canal donde todos los días se mezclaba y multiplicaba la información nueva con la antigua a la fuerza.

La gran idea de la LSTM fue dividir la memoria en dos caminos separados. Por abajo mantuvo el Estado Oculto (h_t) para manejar la memoria rápida de corto plazo del día actual, y por arriba creó una línea recta llamada Estado de Celda (c_t) dedicada exclusivamente a la memoria de largo plazo. Como por esa línea superior la información viaja mediante sumas simples sin sufrir multiplicaciones que la desgasten, la red puede recordar patrones durante cientos de días sin olvidarlos.



Innovación central de Hochreiter & Schmidhuber: La adición lineal (+) en la autopista superior previene la amortiguación del gradiente durante cientos de etapas consecutivas.

Las tres compuertas de la celda LSTM

Para decidir qué recuerdos merecen guardarse en ese canal de largo plazo y cuáles deben descartarse, la celda LSTM utiliza tres compuertas que funcionan como llaves de paso con activación Sigmoide (dando un valor entre 0 para cerrar la llave y 1 para abrirla). La primera es la Compuerta de Olvido (Forget Gate), que mira el dato nuevo de hoy y borra de la memoria antigua aquello que ya dejó de ser útil.

La segunda es la Compuerta de Entrada (Input Gate), que evalúa qué parte de la información nueva que llegó hoy es importante y la suma (+) directamente a la memoria de largo plazo: Nueva Memoria = (Olvido · Memoria Ayer) + (Entrada · Dato Nuevo). Finalmente, la tercera es la Compuerta de Salida (Output Gate), que filtra esa memoria acumulada para entregar la respuesta necesaria en el paso actual.

1. **Compuerta de Olvido (f_t)**
   Filtro sigmoide que decide qué información obsoleta del estado anterior c_(t-1) se borra de la memoria.

2. **Compuerta de Entrada (i_t)**
   Cuantifica qué proporción de la información nueva recibida hoy se suma (+) a la memoria de largo plazo.

3. **Compuerta de Salida (o_t)**
   Filtra y condiciona la memoria acumulada para emitir el estado oculto h_t y la respuesta en el paso actual.

La celda GRU: una alternativa más rápida

Aunque la celda LSTM resolvió con éxito el problema de la memoria a largo plazo, tener dos estados separados y tres compuertas internas hace que tenga cuatro veces más pesos que una RNN simple, por lo que requiere más memoria RAM y más tiempo de entrenamiento. Buscando una opción más ligera, en 2014 Kyunghyun Cho y el equipo de Yoshua Bengio crearon la celda GRU (Gated Recurrent Unit).

La celda GRU simplifica el diseño volviendo a unir la memoria en un solo estado h_t y reduciendo las compuertas a solo dos: una Compuerta de Reinicio y una Compuerta de Actualización (esta última se encarga de olvidar lo viejo e incorporar lo nuevo en un solo movimiento). Gracias a ello, la GRU tiene un 25% menos de parámetros que una LSTM, entrena bastante más rápido y ofrece una precisión muy similar en datasets pequeños o medianos.



Procesamiento y telemetría industrial en el borde: microcontroladores compactos donde el ahorro de memoria de la celda GRU permite inferencia ágil.

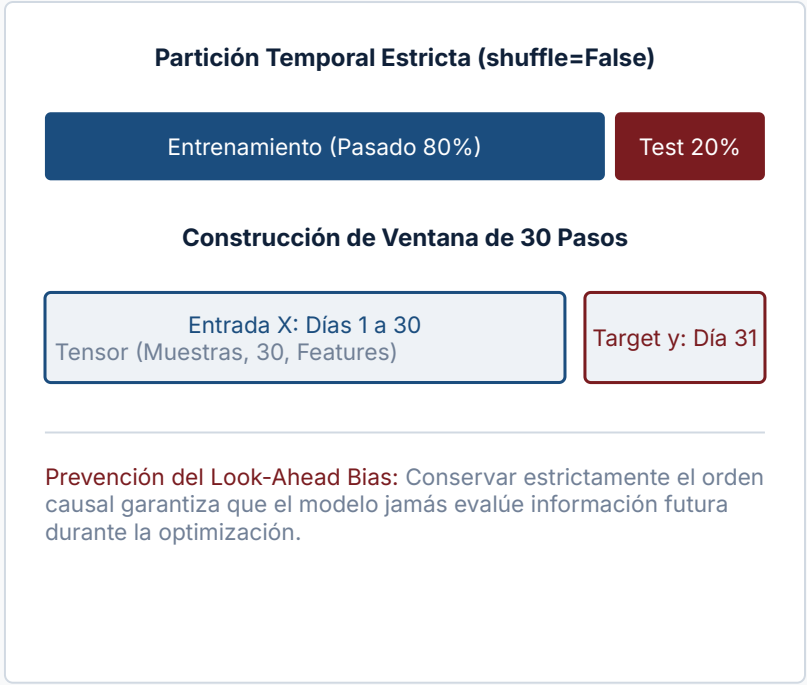
Comparativa entre RNN simple, LSTM y GRU

CRITERIO ANALÍTICO	RNN SIMPLE	LSTM	GRU
Memoria a largo plazo	Muy corta: sufre de amnesia tras 7 a 10 pasos temporales.	Excelente: recuerda cientos de pasos mediante su celda aditiva.	Muy buena: recuerda cientos de pasos con alta retención.
Estados de memoria	1 solo estado recurrente básico (h_t).	2 estados separados: Largo plazo (c_t) y Corto plazo (h_t).	1 estado unificado y protegido acoplando ambos flujos.
Compuertas internas	Ninguna (0 compuertas reguladoras).	3 compuertas: Olvido (f_t), Entrada (i_t) y Salida (o_t).	2 compuertas: Reinicio (r_t) y Actualización (z_t).
Cantidad de pesos	Liviana: 1× pesos (mínimo costo de cómputo).	Pesada: 4× veces más pesos por sus tres compuertas.	Intermedia: 3× pesos (25% más ligera que LSTM).
Cuándo conviene usarla	Solo para ejemplos teóricos cortos y prototipos iniciales.	Series largas y complejas con abundantes datos históricos disponibles.	Proyectos con datos moderados o cuando buscamos entrenar rápido.

Preparación de series de tiempo en Python

Antes de programar una red LSTM en Python para predecir precios o temperaturas, debemos respetar una regla fundamental: jamás barajar las fechas al azar (shuffle=False). A diferencia de las tablas normales, en el tiempo debemos entrenar estrictamente con el pasado (por ejemplo, el primer 80% de los días) y probar con el futuro (el último 20%). Si revolvemos los días, la red verá datos futuros durante el entrenamiento (Look-Ahead Bias) y sus métricas serán falsas.

Una vez separado el pasado del futuro, armamos ventanas deslizantes (Sliding Windows), por ejemplo de 30 días. Tomamos los días del 1 al 30 como entrada (X) para que la red aprenda a predecir el día 31 (y); luego avanzamos un día y tomamos del 2 al 31 para predecir el día 32. De esta manera formamos el bloque tridimensional que exige Keras: (Cantidad de Muestras, 30 Pasos de Tiempo, Número de Variables).



Implementación con Keras en Python

```python
from tensorflow.keras import layers, models

modelo_lstm = models.Sequential([
  # Primera capa LSTM: pasa toda la secuencia a la segunda
  layers.LSTM(64, return_sequences=True,
  input_shape=(30, 1)),
  layers.Dropout(0.2),
  # Segunda capa LSTM: entrega solo el resumen final
  layers.LSTM(32, return_sequences=False),
  layers.Dropout(0.2),
  # Salida de 1 neurona para predecir el valor de manana
  layers.Dense(1)
])

modelo_lstm.compile(optimizer='adam', loss='mse')
```

En este código con TensorFlow y Keras conectamos dos capas LSTM seguidas para darle mayor capacidad al modelo. Un detalle clave que siempre debemos cuidar es el parámetro `return_sequences`: en la primera capa LSTM es obligatorio poner `return_sequences=True` para que entregue la secuencia completa de los 30 pasos a la segunda capa LSTM que la espera abajo.

En cambio, en la segunda capa LSTM ponemos `return_sequences=False` porque después de ella ya viene la neurona densa final, la cual solo necesita el resumen del último paso para calcular el valor de mañana. Además, intercalamos capas de `Dropout(0.2)` para evitar que la red memorice el ruido de la serie y compilamos con la pérdida de error cuadrático medio (mse).

Caso de estudio: Bolsa de Valores en la UNMSM

Para ver cómo se aplican las redes recurrentes en la investigación científica de nuestra facultad, analizamos dos estudios recientes liderados por el Dr. David Mauricio en 2025: Prediction of the Maximum and Minimum Prices of Stocks in the Stock Market Using a Hybrid Model Based on Stacking (Tuesta, Flores & Mauricio, 2025, en la revista *Algorithms*) y Factors, Forecasts, and Simulations of Volatility in the Stock Market (Mansilla-Lopez, Mauricio & Narváez, 2025).

En el mercado de valores, los precios de las acciones cambian de forma no lineal y brusca debido a noticias económicas y movimientos de compra y venta. Para un inversionista no basta con estimar un precio promedio; necesita predecir cuál será el precio Máximo (High) y el precio Mínimo (Low) que alcanzará la acción en el día para saber a qué valor comprar y dónde fijar su límite de protección contra pérdidas (Stop-Loss).



Dinámica intradiaria del mercado bursátil

Seguimiento analítico de series temporales no lineales para la estimación de cotizaciones extremas y mitigación de riesgo financiero.

Variables e indicadores de la ventana temporal

Si alimentamos una red LSTM únicamente con el precio de cierre del día anterior, le faltará contexto para anticipar cambios de tendencia. Por eso, en el estudio de Tuesta y Mauricio (2025), cada ventana temporal se construye con múltiples variables diarias: los cinco datos básicos de cotización (precio de apertura, máximo, mínimo, cierre y volumen de acciones negociadas).

A esos datos básicos se les suman indicadores técnicos que resumen la fuerza del mercado en las últimas semanas, como las medias móviles simples y exponenciales (SMA y EMA), el índice de fuerza relativa (RSI, que detecta si una acción subió demasiado rápido y está por corregir) y las Bandas de Bollinger, además de factores macroeconómicos como tasas de interés y tipo de cambio.



Monitor bursátil con curvas históricas de precios, medias móviles y Bandas de Bollinger integradas para la parametrización de la ventana temporal.

Modelo híbrido con LSTM y Stacking

Los investigadores observaron que en días estables un tipo de modelo predice muy bien, pero en días de alta turbulencia financiera otro modelo responde mejor. Para no depender de un único algoritmo aislado, diseñaron un modelo híbrido basado en apilamiento (Stacking) organizado en dos niveles que trabajan como un comité de especialistas.

En el primer nivel colocan a trabajar en paralelo a redes recurrentes con memoria (LSTM y GRU) junto con regresores de soporte vectorial (SVR) y árboles potenciados, de modo que cada uno analiza las ventanas históricas y lanza su pronóstico. En el segundo nivel, un modelo combinador (Meta-Learner) recibe todas esas estimaciones y aprende a ponderarlas para emitir un pronóstico final mucho más estable y preciso.

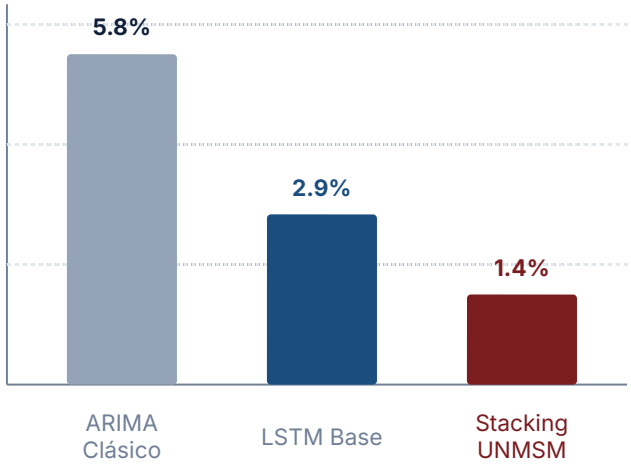


Diversidad algorítmica (Tuesta & Mauricio, 2025): Los modelos secuenciales capturan la inercia temporal, mientras que los modelos de soporte vectorial corrigen fluctuaciones bruscas no lineales.

Evaluación de resultados: RMSE y MAPE

Como en series de tiempo predecimos valores numéricos continuos (soles o dólares) y no categorías fijas, evaluamos el rendimiento midiendo qué tan cerca estuvo la curva predicha de la curva real. Para ello se utilizan la Raíz del Error Cuadrático Medio (RMSE), que castiga los desvíos grandes en picos de precios, y el Error Porcentual Absoluto Medio (MAPE), que indica en qué porcentaje se equivocó el modelo en promedio.

En las pruebas experimentales del artículo, mientras que los métodos econométricos tradicionales como ARIMA tuvieron un error cercano al 5.8% y una red LSTM trabajando sola tuvo alrededor de 2.9% de error, el modelo híbrido con Stacking de la UNMSM redujo el error MAPE a solo 1.4% – 2.0%, alcanzando una correlación (R^2) de 0.98 entre el precio predicho y el real.



Error porcentual MAPE validado experimentalmente (Tuesta & Mauricio, 2025).

Simulación de volatilidad y aplicación agrícola

En el estudio complementario de Mansilla-Lopez, Mauricio y Narváez (2025), estos modelos se integran con simulaciones de volatilidad para proyectar bandas de riesgo. Esto permite a bancos, cajas municipales y administradoras de fondos de pensiones (AFP) evaluar escenarios de caída del mercado y proteger los fondos de los ahorristas bajo supervisión de analistas humanos (cumpliendo con la Ley N.° 31814).

Además, lo más valioso para nosotros como ingenieros de sistemas es que esta misma estructura de ventanas deslizantes con redes LSTM se aplica directamente a otros problemas críticos del Perú. Si en lugar de meter 30 días de precios bursátiles ingresamos 30 días de temperatura, humedad y viento de estaciones del SENAMHI en Puno o Junín, el modelo predice heladas agrícolas con dos días de anticipación para proteger las cosechas de papa.



Transferencia tecnológica: de mercados financieros a seguridad alimentaria

Monitoreo agroclimático en cultivos de papa en el altiplano y valles interandinos. Redes LSTM alimentadas con registros históricos del SENAMHI anticipan heladas con 48 horas de antelación.

De las redes LSTM a los Transformers

Para conectar el cierre de esta Unidad IV con la Unidad V (IA Generativa), vale la pena preguntarse: si las redes LSTM y GRU ya tenían tan buena memoria a largo plazo, ¿por qué en 2017 se crearon los Transformers, que hoy son la base de modelos como ChatGPT? La respuesta está en la velocidad de entrenamiento cuando se manejan millones de textos.

Como una red LSTM procesa los datos en fila india (primero debe terminar de calcular el día 1 para recién pasar al día 2, y luego al día 3), las tarjetas gráficas GPU no pueden calcular todos los pasos al mismo tiempo. En 2017, la arquitectura Transformer reemplazó ese recorrido paso a paso por el Mecanismo de Atención, que mira toda la secuencia de un solo vistazo en paralelo; sin embargo, para series numéricas de clima, finanzas y sensores, las redes LSTM y GRU siguen siendo hoy una de las herramientas más eficientes y utilizadas.



Comparativa de las arquitecturas de la Unidad IV

DIMENSIÓN / CRITERIO	SEMANA 9 **Perceptrón Multicapa (MLP)**	SEMANA 10 **Red Neuronal Convolucional (CNN)**	SEMANA 11 **Redes Recurrentes (RNN, LSTM, GRU)**
Naturaleza de los Datos Estructura y supuestos de independencia entre registros.	Diseñado para tablas de datos donde cada fila es independiente.	Diseñada para imágenes, fotografías médicas y video en 2D.	Diseñadas para series de tiempo y secuencias donde el orden del pasado importa.
Casos de Estudio Ejemplos prototípicos abordados en las sesiones de laboratorio.	Ejemplos: Datos socioeconómicos y notas de alumnos.	Ejemplos: Fotos de retina para glaucoma u hojas de papa.	Ejemplos: Precios de bolsa, clima o señales de ECG.
Arquitectura y Explicación Topología de capas y herramientas de interpretabilidad.	Utiliza capas ocultas densas con activación ReLU. Se explica con gráficos SHAP.	Utiliza filtros locales de 3×3 y capas de Max Pooling. Se explica con mapas de calor Grad-CAM.	Utilizan estado de celda con compuertas. Estructuración con ventanas deslizantes cronológicas.

Preguntas de repaso y discusión

1. ¿Por qué en un problema de series de tiempo está prohibido barajar los datos al azar (shuffle=True)?

Porque mezclaríamos fechas futuras dentro del entrenamiento, haciendo que el modelo vea el futuro por adelantado (Look-Ahead Bias) y colapse en situaciones reales.

2. ¿Por qué la RNN simple olvida lo que pasó hace varias semanas y cómo lo corrige la celda LSTM?

La RNN simple olvida porque al retroceder multiplica valores menores a 1 hasta que el gradiente se anula; la LSTM lo corrige creando la autopista de celda (c_t) donde la información se suma (+) sin desgastarse.

3. ¿Qué función cumplen la Compuerta de Olvido y la Compuerta de Entrada en una LSTM?

La compuerta de olvido decide qué porcentaje del recuerdo antiguo ya no sirve y debe descartarse, mientras que la compuerta de entrada filtra qué dato nuevo de hoy vale la pena incorporar a la memoria.

4. ¿Por qué al poner dos capas LSTM seguidas en Keras la primera lleva return_sequences=True y la segunda False?

Porque la primera debe pasarle los 30 pasos completos a la segunda capa LSTM, mientras que la segunda solo entrega el resumen final a la neurona densa.

Conclusiones

1. Cuando trabajamos con series temporales o secuencias, las redes estáticas no son suficientes; necesitamos arquitecturas recurrentes capaces de mantener un estado interno de memoria que conecte el pasado con el presente.

2. Las celdas LSTM y GRU superan el desvanecimiento del gradiente de las RNN simples mediante el uso de compuertas inteligentes y un canal de memoria aditivo, logrando retener patrones a lo largo de cientos de pasos.

3. El entrenamiento correcto de una serie temporal en Python requiere dividir el pasado y el futuro sin barajar las fechas, estructurar ventanas deslizantes en formato 3D y evaluar los resultados con métricas de error como RMSE y MAPE.

4. Los estudios desarrollados en la UNMSM (Tuesta & Mauricio, 2025; Mansilla-Lopez & Mauricio, 2025) demuestran que combinar redes LSTM mediante Stacking reduce el error de pronóstico bursátil por debajo del 1.5%, con aplicación directa en finanzas y climatología peruana bajo la Ley N.° 31814.

Referencias bibliográficas clave: Elman (1990), Hochreiter & Schmidhuber (1997), Cho et al. (2014), Goodfellow et al. (2016), Chollet (2021), Tuesta & Mauricio (2025), Mansilla-Lopez & Mauricio (2025), Ley N.° 31814.